

Backtracking DSA Handbook

A Complete, Practical Guide to Backtracking Patterns in Python

PART 1: BACKTRACKING FUNDAMENTALS

- 1.1 What is Backtracking?
- 1.2 Core Terminology
- 1.3 The Backtracking Template
- 1.4 Backtracking vs Brute Force vs DP
- 1.5 How to Read This Handbook

PART 2: PATTERNS

- 2.1 Subsets & Combinations
- 2.2 Permutations
- 2.3 Combination Sum (Choice With/Without Repetition)
- 2.4 Partitioning Problems
- 2.5 Grid / Maze Backtracking
- 2.6 Constraint Satisfaction (N-Queens, Sudoku)
- 2.7 String Backtracking
- 2.8 Pruning, Bounding & Bitmask Optimization

PART 3: QUICK REFERENCE CHEAT SHEET

Decision Guide — “How Do I Shape the Backtracking?”

PART 1: FUNDAMENTALS

1.1 What is Backtracking?

Backtracking is a refinement of recursion (specifically, tree/multiple recursion) for exploring the full space of possible choices: at each step you make a choice, recurse to explore everything that follows from it, and then UNDO that choice before trying the next one. This "choose → explore → unchoose" cycle is what distinguishes backtracking from plain recursive branching — it reuses the same working state (array, string, board) across branches instead of copying it, and abandons ("prunes") a branch as soon as it can't possibly lead to a valid answer.

1.2 Core Terminology

State / Partial Solution	The choices made so far on the current path (e.g., the subset built up to this point, the board filled so far).
Choice / Candidate	One option available at the current decision point (e.g., which number to place next, which element to include).
State Space Tree	The tree of all possible sequences of choices; backtracking performs a DFS over this tree.
Pruning	Cutting off a branch early once you can prove it cannot lead to a valid/optimal solution, instead of exploring it fully.
Constraint Check / Feasibility	The test applied before or during a choice to see if it keeps the partial solution valid.
Un-choose (Backtrack Step)	Undoing the most recent choice (removing it from the state) before returning control to the parent call, so the next sibling choice starts clean.
Base Case / Goal Test	The condition marking a complete, valid solution (e.g., all positions filled, path length reached).
Bounding Function	A stronger, often numeric, check used to prune large parts of the search space early (e.g., "remaining sum can't possibly reach target").

1.3 The Backtracking Template

Nearly every backtracking solution follows the same skeleton. Internalizing this template is more valuable than memorizing individual problems:

```
def backtrack(state, choices_remaining):
    if is_goal(state):                # 1. Base case: state is a complete valid solution
        record_solution(state)
        return                       # (or `return True` for existence-only problems)

    for choice in candidates(state, choices_remaining):
        if not is_valid(state, choice): # 2. Prune invalid choices before recursing
            continue

        make_choice(state, choice)     # 3. CHOOSE: add the choice to the state
        backtrack(state, choices_remaining) # 4. EXPLORE: recurse with the updated state
        undo_choice(state, choice)     # 5. UN-CHOOSE: remove the choice (the "backtrack" step)
```

1.4 Backtracking vs Brute Force vs DP

vs Brute Force	Brute force generates and checks every possibility independently; backtracking builds possibilities incrementally and abandons a branch the moment it becomes invalid, avoiding wasted work on doomed prefixes.
vs Plain (Tree) Recursion	Plain branching recursion (see the Recursion handbook, 2.2) usually keeps exploring and combining results; backtracking specifically mutates and restores shared state, and actively prunes infeasible branches.
vs Dynamic Programming	DP applies when subproblems overlap and you want one optimal/aggregate answer; backtracking is for enumerating or finding ANY/ALL valid full solutions where the "path so far" (order, exact set of choices) matters, so subproblems generally do NOT overlap in a reusable way.
When results are exponential	If a problem inherently asks for "all subsets / all permutations / all valid boards", the output itself is exponential — backtracking is optimal because you can't avoid producing all of them, but pruning still saves time by skipping invalid branches early.

1.5 How to Read This Handbook

For every pattern below you will get, in this exact order: (1) What it's for, (2) Use Cases, (3) Limitations, (4) Time & Space Complexity, (5) Approach (short), (6) Keywords & Constraints that hint you should use this pattern, (7) Fully commented Python code, and (8) Practice Questions with a short reasoning of why the pattern fits.

PART 2: PATTERNS & ALGORITHMS

2.1 Subsets & Combinations

What it's for: Generates every subset, or every fixed-size combination, of a set of elements by deciding at each index whether to include it (subsets) or by choosing k elements moving forward only (combinations).

Use Cases:

- Power set generation for a set of items.
- Choosing k items out of n without regard to order (committees, lottery-style problems).
- Feature/flag combinations for testing, subset-sum style enumeration.
- Building block for more constrained problems (combination sum, partitioning) layered on top.

Limitations:

- Output size is inherently exponential (2^n subsets, $C(n,k)$ combinations) — no algorithm can avoid producing all of them if all are requested.
- Without moving a "start index" forward for combinations, you risk generating the same combination multiple times in different orders.
- Including duplicate elements in the input requires an explicit dedup step (sort + skip same-value siblings) or you'll emit duplicate subsets.

Time Complexity: $O(2^n \times n)$ for all subsets; $O(C(n,k) \times k)$ for fixed-size combinations (the $\times n/\times k$ accounts for copying each result) **Space Complexity:** $O(n)$ recursion depth plus $O(2^n \times n)$ or $O(C(n,k) \times k)$ to store all results

Approach (short): At each index, branch into "skip this element" and "take this element" (subsets), or iterate candidates from a moving start index forward only, adding one at a time and recursing (combinations) before removing it and trying the next candidate.

Keywords & Constraints: "all subsets", "power set", "choose k of n ", "combinations", $n \leq 20$ -25 typically (since output is exponential).

```
def subsets(nums):
    """All subsets via include/exclude branching."""
    result, current = [], []
    def backtrack(i):
        if i == len(nums):
            result.append(current[:])          # snapshot -- current will keep mutating
            return
        # exclude nums[i]
        backtrack(i + 1)
        # include nums[i]
        current.append(nums[i])
        backtrack(i + 1)
        current.pop()                          # UN-CHOOSE before returning
    backtrack(0)
    return result

def combinations(n, k):
    """All ways to choose k numbers from 1..n."""
    result, current = [], []
    def backtrack(start):
        if len(current) == k:
            result.append(current[:])
            return
        for num in range(start, n + 1):
            current.append(num)                # only look FORWARD to avoid duplicate orderings
            backtrack(num + 1)                 # CHOOSE
            current.pop()                       # EXPLORE
            current.pop()                       # UN-CHOOSE
    backtrack(1)
    return result
```

```

return result

def subsets_with_dup(nums):
    """All subsets when input may contain duplicates -- sort + skip same-value siblings."""
    nums.sort()
    result, current = [], []
    def backtrack(start):
        result.append(current[:])
        for i in range(start, len(nums)):
            if i > start and nums[i] == nums[i - 1]:
                continue # skip duplicate choice AT THE SAME DEPTH
            current.append(nums[i])
            backtrack(i + 1)
            current.pop()
    backtrack(0)
    return result

```

Practice Questions — Subsets & Combinations

Q1. Subsets (Power Set) Return all possible subsets of a set of distinct integers. *Approach:* Include/exclude branching at each index; 2^n leaves, one snapshot of `current` per leaf.

Q2. Subsets II (With Duplicates) Return all unique subsets when input may contain duplicates. *Approach:* Sort first, then at each recursion depth skip a candidate equal to the previous sibling to avoid emitting the same subset twice.

Q3. Combinations Return all combinations of k numbers chosen from 1 to n. *Approach:* Move a start index forward only, so each combination is generated exactly once in increasing order.

Q4. Combination Sum III Find all combinations of k numbers (1-9, no repeats) that sum to n. *Approach:* Same forward-moving combinations template, with an added bounding check to prune once the running sum exceeds n or remaining slots can't possibly reach it.

Q5. Letter Case Permutation Generate all strings by changing letters to upper/lower case. *Approach:* At each alphabetic character, branch into "lowercase" and "uppercase" (digits pass through unchanged) — a subsets-style include/exclude but over case choice instead of presence.

Q6. Partition into K Equal Sum Subsets Determine if an array can be split into k subsets of equal sum. *Approach:* Backtrack by assigning each number to one of k running "bucket sums", pruning any bucket that would exceed the target sum — builds directly on subset-style choice branching.

2.2 Permutations

What it's for: Generates every possible ordering of a set of elements, where order matters (unlike combinations). Built by picking one unused element at a time for each position until all positions are filled.

Use Cases:

- Arranging n distinct items in every possible order.
- Generating all possible passwords/sequences from a fixed alphabet of characters.
- Traveling Salesman brute-force baseline (try every city ordering).
- Anagram generation, scheduling orderings with no additional constraints.

Limitations:

- Output size is $n!$ — grows far faster than 2^n , so this is only feasible for small n (typically $\leq 10-11$).
- Requires tracking which elements are "used" (a boolean array or set) — forgetting to un-mark on backtrack is a very common bug.
- Duplicate elements in the input need sorting + same-depth-skip logic, exactly like subsets with duplicates, or you'll emit repeated permutations.

Time Complexity: $O(n! \times n)$ — $n!$ permutations, each taking $O(n)$ to build/copy **Space Complexity:** $O(n)$ recursion depth and "used" tracking, plus $O(n! \times n)$ to store all results

Approach (short): At each position in the result, try every not-yet-used element: mark it used, append it, recurse to fill the next position, then un-mark and remove it before trying the next candidate.

Keywords & Constraints: "all permutations", "all orderings/arrangements", "anagrams", $n \leq 8-11$ typically (since output is $n!$).

```
def permutations(nums):
    """All orderings of distinct elements."""
    result, current = [], []
    used = [False] * len(nums)
    def backtrack():
        if len(current) == len(nums):
            result.append(current[:])
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            used[i] = True
            current.append(nums[i])
            backtrack()
            current.pop()
            used[i] = False
    backtrack()
    return result

def permutations_with_dup(nums):
    """All UNIQUE orderings when input may contain duplicate values."""
    nums.sort()
    result, current = [], []
    used = [False] * len(nums)
    def backtrack():
        if len(current) == len(nums):
            result.append(current[:])
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                continue
            used[i] = True
            current.append(nums[i])
            backtrack()
            current.pop()
            used[i] = False
    backtrack()
    return result
```

```

        backtrack()
        current.pop()
        used[i] = False
    backtrack()
    return result

```

```

def next_permutation_via_backtrack_check(nums, target):
    """Illustrative: search permutations in lexicographic order until target is found (small n only)."""
    for perm in permutations(sorted(nums)):
        if perm == target:
            return perm
    return None

```

Practice Questions — Permutations

Q1. Permutations Return all possible orderings of a distinct-integer array. *Approach:* Track used[] to avoid reusing an element in the same path; choose an unused element, recurse, then un-choose and try the next.

Q2. Permutations II (With Duplicates) Return all unique permutations when input contains duplicates. *Approach:* Sort first, and at each position skip a duplicate value unless its identical predecessor was already used on this path (the classic `!used[i-1]` guard).

Q3. Next Permutation (Conceptual / Brute Force) Find the lexicographically next permutation of an array. *Approach:* For very small n, generating all permutations in order via backtracking and finding the current one's successor illustrates the concept, though the true O(n) two-pointer algorithm is preferred in practice.

Q4. Letter Combinations of a Phone Number Given digits 2-9, return all letter combinations they could represent. *Approach:* For each digit position, branch over that digit's mapped letters and recurse to the next digit — permutation-style choice, but candidates differ per position instead of a shared pool.

Q5. N-Queens (Permutation View) Place n queens so no two attack each other. *Approach:* Since each queen must occupy a distinct column, this can be framed as permuting column assignments across rows and pruning any assignment that creates a diagonal/column attack.

Q6. Beautiful Arrangement Count permutations of 1..n where position i is divisible by value or vice versa. *Approach:* Standard permutation backtracking with an added feasibility check (divisibility) applied BEFORE placing each number — shows how a permutation template gains a constraint check without changing its shape.

2.3 Combination Sum (Choice With/Without Repetition)

What it's for: Finds all combinations of numbers that sum to a target, where the same number may be reused unlimited times, reused a limited number of times, or not reused at all — the core distinction being whether the recursive call advances the start index or keeps it the same.

Use Cases:

- Coin/denomination combinations that reach an exact total.
- Finding all ways to make change, form a sum from a menu of item prices, etc.
- Any "which subset/multiset of items reaches exactly this target" enumeration.
- A natural stepping stone toward Unbounded/0-1 Knapsack in the DP handbook once only the COUNT or MIN/MAX (not every combination) is needed.

Limitations:

- Allowing unlimited reuse without sorting and pruning can revisit the same combination in different orders — always move forward from a start index, never restart from 0.
- Needs a strict pruning rule (running sum > target → stop) or it degenerates into brute-force enumeration of far too many partial sums.
- If only the count or feasibility (not every explicit combination) is needed, DP is asymptotically faster — backtracking is for when you need the actual combinations.

Time Complexity: Exponential in the worst case, roughly $O(\text{target}^n)$ bounded by pruning; highly input-dependent

Space Complexity: $O(\text{target}/\text{min_candidate})$ recursion depth, plus space for all found combinations

Approach (short): Sort candidates. At each call, try each candidate from the current start index onward: if reuse is allowed, recurse with the SAME start index; if not, recurse with start index + 1. Prune the moment the running sum exceeds the target.

Keywords & Constraints: "combinations that sum to target", "may be used multiple times", "each number used once", target and candidate count both fairly small (target ≤ a few hundred, candidates ≤ 30-ish).

```
def combination_sum(candidates, target):
    """Numbers may be reused unlimited times."""
    candidates.sort()
    result, current = [], []
    def backtrack(start, remaining):
        if remaining == 0:
            result.append(current[:])
            return
        for i in range(start, len(candidates)):
            if candidates[i] > remaining:
                break
            current.append(candidates[i])
            backtrack(i, remaining - candidates[i])  # SAME index i -> reuse allowed
            current.pop()
    backtrack(0, target)
    return result
```

```
def combination_sum_ii(candidates, target):
    """Each number may be used at most ONCE; input may contain duplicates."""
    candidates.sort()
    result, current = [], []
    def backtrack(start, remaining):
        if remaining == 0:
            result.append(current[:])
            return
        for i in range(start, len(candidates)):
            if i > start and candidates[i] == candidates[i - 1]:
                continue
            if candidates[i] > remaining:
                break
            current.append(candidates[i])
            backtrack(i + 1, remaining - candidates[i])
            current.pop()
```



```

        break
    current.append(candidates[i])
    backtrack(i + 1, remaining - candidates[i])    # i + 1 -> no reuse
    current.pop()
    backtrack(0, target)
    return result

```

Practice Questions — Combination Sum (Choice With/Without Repetition)

Q1. Combination Sum Find all combinations that sum to target, numbers may repeat. *Approach:* Recurse with the SAME start index when reusing a candidate; sort candidates first so you can break early once a candidate exceeds the remaining target.

Q2. Combination Sum II Find all combinations summing to target, each number used at most once, input has duplicates. *Approach:* Sort, advance start index by 1 (no reuse), and additionally skip duplicate siblings at the same recursion depth to avoid duplicate combinations.

Q3. Combination Sum III Find combinations of exactly k numbers from 1-9 summing to n. *Approach:* Same forward-index template plus two simultaneous bounds: stop once k numbers are chosen AND check the sum equals n exactly.

Q4. Factor Combinations Find all ways to write a number as a product of factors ≥ 2 . *Approach:* Structurally identical to Combination Sum with unlimited reuse, but the "remaining" quantity is divided rather than subtracted, and factors must stay $\geq$ the previous factor to avoid order duplicates.

Q5. Combinations that Sum to Target with Limited Supply Each candidate has a limited count available (e.g., at most 3 of each coin). *Approach:* Extend the reuse case with a per-candidate remaining-count parameter that decrements on each use and is restored on backtrack.

Q6. Minimum Number of Coins (Enumerate then Pick Best) Among all valid coin combinations reaching a target, find the one using fewest coins. *Approach:* Generate combinations with the unlimited-reuse template while tracking the best (shortest) `current` found so far — illustrates why, once only the MINIMUM matters, switching to DP (coin change) is far more efficient than enumerating every combination.

2.4 Partitioning Problems

What it's for: Splits a sequence (string or array) into pieces that each satisfy some property, exploring every possible way to place the "cut points" between elements.

Use Cases:

- Palindrome partitioning — split a string so every piece is a palindrome.
- Partitioning an array into subsets/segments with equal sum or balanced properties.
- Word Break style segmentation into valid dictionary words.
- Splitting a sequence into the minimum/maximum number of valid contiguous chunks.

Limitations:

- Number of possible partitions of a length- n sequence is $2^{(n-1)}$ (one binary choice per gap) — exponential, so only tractable for modest n unless paired with memoization/DP.
- A naive re-check of "is this substring/subarray valid" at every cut can add an extra $O(n)$ or $O(n^2)$ factor — precomputing validity (e.g., a palindrome table) is a common optimization.
- Easy to conflate "partition" (contiguous pieces covering the whole sequence in order) with "subset" (any selection, order doesn't matter) — they use different loop structures.

Time Complexity: $O(2^n \times n)$ or $O(2^n \times \text{validation_cost})$ in the worst case, before any memoization/precomputation **Space Complexity:** $O(n)$ recursion depth plus space for all valid partitions found

Approach (short): At each starting index, try every possible "next cut" position; if the piece between start and cut is valid, recurse on the remainder starting right after the cut, and undo before trying the next cut length.

Keywords & Constraints: "partition into", "split into pieces where each is X", "segment the string/array", $n \leq 15-20$ for pure backtracking without memoization.

```
def is_palindrome(s):
    return s == s[::-1]

def palindrome_partition(s):
    """All ways to split s so every piece is a palindrome."""
    result, current = [], []
    def backtrack(start):
        if start == len(s):
            result.append(current[:])
            return
        for end in range(start + 1, len(s) + 1):
            piece = s[start:end]
            if is_palindrome(piece):
                current.append(piece)
                backtrack(end)
                current.pop()
    backtrack(0)
    return result

def word_break_all(s, word_dict):
    """All ways to segment s into words from word_dict."""
    result, current = [], []
    def backtrack(start):
        if start == len(s):
            result.append(" ".join(current))
            return
        for end in range(start + 1, len(s) + 1):
            piece = s[start:end]
            if piece in word_dict:
                current.append(piece)
                backtrack(end)
                current.pop()
    backtrack(0)
    return result
```

```

def partition_equal_subsets(nums, k):
    """Partition nums into k groups of equal sum (existence check)."""
    total = sum(nums)
    if total % k != 0:
        return False
    target = total // k
    nums.sort(reverse=True)          # try larger numbers first -> prunes faster
    buckets = [0] * k
    def backtrack(i):
        if i == len(nums):
            return True              # all numbers placed within bucket limits
        for b in range(k):
            if buckets[b] + nums[i] <= target:
                buckets[b] += nums[i]    # CHOOSE bucket b
                if backtrack(i + 1):
                    return True
                buckets[b] -= nums[i]    # UN-CHOOSE
            if buckets[b] == 0:          # avoid trying more empty buckets (symmetry prune)
                break
        return False
    return backtrack(0)

```

Practice Questions — Partitioning Problems

Q1. Palindrome Partitioning Return all ways to split a string so every substring is a palindrome. *Approach:* Try every cut point from the current start; only recurse into a piece if it's a palindrome, pruning non-palindromic prefixes immediately.

Q2. Word Break II Return all sentences formed by segmenting a string into dictionary words. *Approach:* Same "try every next cut" template as palindrome partitioning, but validity check is dictionary membership instead of palindrome-ness.

Q3. Partition to K Equal Sum Subsets Determine if an array can be divided into k subsets of equal sum. *Approach:* Backtrack by placing each number into one of k running bucket sums, pruning a bucket the moment it would exceed the target and skipping symmetric empty-bucket attempts.

Q4. Partition Array into Consecutive Subsequences Split an array into groups of consecutive integers of length ≥ 3 . *Approach:* Greedy-with-backtracking: try extending an existing run or starting a new one at each element, backtracking if a valid full partition can't be completed.

Q5. Restore IP Addresses Insert dots into a digit string to form all valid IP addresses. *Approach:* Partition the string into exactly 4 pieces, trying every cut length (1-3 digits) at each step and pruning pieces that are out of the 0-255 range or have invalid leading zeros.

Q6. Minimum Cuts for Palindrome Partitioning Find the minimum number of cuts to partition a string into palindromes. *Approach:* Backtracking enumerates all partitions to explain the search space, but for the MINIMUM count specifically this becomes a DP problem (min cuts) once you only need the best count, not every partition.

2.5 Grid / Maze Backtracking

What it's for: Explores paths through a 2D grid by moving to adjacent cells, marking cells as visited before recursing and un-marking them on the way back so other paths can reuse that cell.

Use Cases:

- Word Search — find if a word can be traced through adjacent grid cells.
- Rat in a Maze — find all paths from start to destination avoiding blocked cells.
- Counting/enumerating all paths in a grid subject to movement constraints.
- Puzzle games where a piece/character explores a board and must backtrack from dead ends.

Limitations:

- Must explicitly mark cells visited (in-place marker or a separate visited set/grid) AND unmark them on backtrack, or you'll get infinite loops or miss valid paths that reuse a cell from a different direction.
- Without pruning (e.g., early character mismatch in Word Search), worst-case time is exponential in the number of cells (up to 4^L (path length) for 4-directional movement).
- Recursion depth can reach rows×cols for a maze that snakes through every cell — large grids may need an iterative stack-based DFS instead.

Time Complexity: $O(\text{rows} \times \text{cols} \times 4^L)$ in the worst case, where L is the path/word length being searched **Space Complexity:** $O(L)$ recursion depth (plus $O(\text{rows} \times \text{cols})$ for a visited grid if not marking in-place)

Approach (short): From the current cell, try each of the (usually 4) neighboring directions: if the neighbor is in-bounds, unvisited, and satisfies the problem's constraint, mark it visited, recurse, then unmark it before trying the next direction.

Keywords & Constraints: "word search", "maze", "grid path", "rat in a maze", "number of paths avoiding obstacles", grid dimensions typically small ($\leq 10 \times 10$ to 12×12) for backtracking to be fast enough.

```
def exist(board, word):
    """Word Search: can `word` be traced through adjacent (up/down/left/right) cells?"""
    rows, cols = len(board), len(board[0])

    def backtrack(r, c, i):
        if i == len(word):
            return True
        if r < 0 or r >= rows or c < 0 or c >= cols or board[r][c] != word[i]:
            return False
        temp, board[r][c] = board[r][c], '#'
        found = (backtrack(r + 1, c, i + 1) or backtrack(r - 1, c, i + 1) or
                 backtrack(r, c + 1, i + 1) or backtrack(r, c - 1, i + 1))
        board[r][c] = temp
        return found

    return any(backtrack(r, c, 0) for r in range(rows) for c in range(cols))

def rat_in_a_maze(maze):
    """All paths from (0,0) to (n-1,n-1) moving in 4 directions, avoiding blocked cells."""
    n = len(maze)
    result = []
    visited = [[False] * n for _ in range(n)]

    def backtrack(r, c, path):
        if r == n - 1 and c == n - 1:
            result.append(path)
            return
        for dr, dc, move in [(1, 0, 'D'), (-1, 0, 'U'), (0, 1, 'R'), (0, -1, 'L')]:
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < n and maze[nr][nc] == 1 and not visited[nr][nc]:
```

```

        visited[nr][nc] = True                # CHOOSE
        backtrack(nr, nc, path + move)        # EXPLORE
        visited[nr][nc] = False              # UN-CHOOSE

    if maze[0][0] == 1:
        visited[0][0] = True
        backtrack(0, 0, "")
    return result

```

Practice Questions — Grid / Maze Backtracking

Q1. Word Search Determine if a word can be found by tracing adjacent cells in a grid. *Approach:* DFS/backtrack from every starting cell, marking cells visited in-place and restoring them on backtrack; prune the instant a character doesn't match.

Q2. Word Search II (Multiple Words via Trie) Find all dictionary words present in the grid. *Approach:* Same grid backtracking as Word Search, but candidates are driven by a Trie built from all target words so many words are pruned together at shared prefixes.

Q3. Rat in a Maze (All Paths) Find all paths from top-left to bottom-right avoiding blocked cells. *Approach:* Standard 4-direction grid backtracking with an explicit visited grid, building the path string as you go and popping the last move on backtrack.

Q4. Number of Paths with Obstacles (Enumerate) Count/list paths from start to end where certain cells are blocked. *Approach:* Same maze template restricted to right/down moves only, pruning on obstacle cells — contrast with the DP version (see DP handbook) once only the COUNT is needed.

Q5. Unique Paths III Find paths that visit every non-obstacle cell exactly once. *Approach:* Grid backtracking with an added global constraint: track how many empty cells remain unvisited and only count a path as valid when it reaches the end AND all empty cells have been visited.

Q6. Flood Fill (Backtracking View) Change the color of a connected region starting from a cell. *Approach:* DFS backtracking style traversal into same-colored neighbors, marking visited cells as you recolor them — no un-choose needed here since the recoloring itself prevents revisiting.

2.6 Constraint Satisfaction (N-Queens, Sudoku)

What it's for: Fills in a structured board (queens on a chessboard, digits in a Sudoku grid) one position at a time, trying every legal value for that position and backtracking whenever no legal value exists, until the entire board is validly filled.

Use Cases:

- N-Queens — place n queens so no two share a row, column, or diagonal.
- Sudoku solving — fill a 9×9 grid respecting row/column/box uniqueness.
- Graph coloring — assign colors to vertices so no adjacent vertices share a color.
- General Constraint Satisfaction Problems (CSPs): scheduling, map coloring, logic puzzles.

Limitations:

- Search space is enormous in the worst case (e.g., naive N-Queens is $O(n^n)$ before pruning) — correctness of the pruning/constraint checks is essential for feasible runtime.
- Constraint checks (e.g., "is this diagonal already attacked?") done naively can add an $O(n)$ factor per placement; tracking attacked columns/diagonals with sets makes checks $O(1)$.
- Sudoku-style problems can still be slow on adversarial/near-empty boards even with constraint checks — real solvers add heuristics like "fill the most-constrained cell first" (not covered here, but good to know exists).

Time Complexity: Highly pruning-dependent; N-Queens with column/diagonal tracking is roughly $O(n!)$ in the worst case but fast in practice for $n \leq 12-15$; Sudoku is technically NP-hard but fast in practice for standard 9×9 puzzles

Space Complexity: $O(n)$ for the board/position stack, plus $O(n)$ for tracking used rows/columns/diagonals or $O(1)$ extra for in-place board checks

Approach (short): Move to the next empty position (next row for N-Queens, next empty cell for Sudoku). Try every legal value for that position (checking row/column/diagonal or row/column/box constraints); place it, recurse to the next position, and remove it if the recursive call fails to complete the board.

Keywords & Constraints: "place n queens", "no two attack each other", "solve the sudoku", "valid board configuration", $n \leq 12-15$ for N-Queens, fixed 9×9 for Sudoku.

```
def solve_n_queens(n):
    """All ways to place n non-attacking queens, one per row."""
    result = []
    cols, diag1, diag2 = set(), set(), set()      # track attacked columns and both diagonals
    placement = [-1] * n                        # placement[row] = column of queen in that row

    def backtrack(row):
        if row == n:
            result.append(placement[:])
            return
        for col in range(n):
            d1, d2 = row - col, row + col        # diagonal identifiers
            if col in cols or d1 in diag1 or d2 in diag2:
                continue                          # PRUNE: this column/diagonal already attacked
            cols.add(col); diag1.add(d1); diag2.add(d2)  # CHOOSE
            placement[row] = col
            backtrack(row + 1)                    # EXPLORE
            cols.remove(col); diag1.remove(d1); diag2.remove(d2)  # UN-CHOOSE

    backtrack(0)
    return result

def solve_sudoku(board):
    """Fill a 9x9 board in place, respecting row/col/box uniqueness."""
    def is_valid(r, c, val):
        for i in range(9):
            if board[r][i] == val or board[i][c] == val:
                return False
```

```

        box_r, box_c = 3 * (r // 3) + i // 3, 3 * (c // 3) + i % 3
        if board[box_r][box_c] == val:
            return False
    return True

def backtrack():
    for r in range(9):
        for c in range(9):
            if board[r][c] == '.':
                for val in '123456789':
                    if is_valid(r, c, val):
                        board[r][c] = val          # CHOOSE
                        if backtrack():              # EXPLORE
                            return True
                        board[r][c] = '.'          # UN-CHOOSE: this value led nowhere
                return False                       # no value works here -> trigger backtrack upstream
    return True                                   # no empty cells left -> solved

backtrack()
return board

```

Practice Questions — Constraint Satisfaction (N-Queens, Sudoku)

Q1. N-Queens Return all distinct ways to place n non-attacking queens. *Approach:* Place one queen per row, tracking attacked columns and both diagonal directions in sets for $O(1)$ constraint checks before each placement.

Q2. N-Queens II (Count Only) Count the number of distinct solutions instead of listing them. *Approach:* Identical backtracking to N-Queens, just increment a counter at the base case instead of storing the full board — shows how "count solutions" vs "list solutions" barely changes the template.

Q3. Sudoku Solver Fill a partially completed 9x9 Sudoku board. *Approach:* Find the next empty cell, try digits 1-9 checking row/column/box validity, and backtrack (reset to '.') the instant a full solution can't be completed down that branch.

Q4. Valid Sudoku (Check Only) Determine if a given board (not necessarily solvable further) is valid so far. *Approach:* No backtracking needed here — straight validation using the same row/column/box constraint logic, included to contrast a pure CHECK from a full SOLVE.

Q5. Graph Coloring Determine if a graph's vertices can be colored with k colors so no adjacent vertices match. *Approach:* Assign a color to each vertex in turn, checking that no already-colored neighbor shares the candidate color, backtracking when no color works for the current vertex.

Q6. M-Coloring / Map Coloring Count Count the number of valid ways to color a map/graph with m colors. *Approach:* Same graph coloring backtracking, but increment a counter at the base case (all vertices colored) rather than returning at the first success — explores the full solution space instead of stopping early.

2.7 String Backtracking

What it's for: Builds strings character-by-character (or token-by-token) under a set of formation rules, branching over legal next characters and backtracking when a partial string can no longer lead to a valid result.

Use Cases:

- Generating all valid combinations of balanced parentheses.
- Letter combinations from phone-number-style digit mappings.
- Building all valid IP address segmentations, restoring valid expressions with operators inserted.
- Generating strings matching a pattern/constraint (e.g., all binary strings with no two consecutive 1s).

Limitations:

- Output can be very large (e.g., Catalan-number growth for balanced parentheses) — pruning invalid prefixes early (not just at the end) is essential for performance.
- String concatenation at every level can add an extra $O(n)$ factor per node if not using a mutable list/char-array accumulator that's appended to and popped from.
- Constraints often depend on COUNTS so far (open vs. close parens used) rather than just the immediate previous character — make sure to thread the right counters through the recursion.

Time Complexity: Varies widely by problem; often $O(2^n)$ to $O(\text{Catalan}(n))$ before pruning, reduced significantly by validity checks applied during construction rather than after **Space Complexity:** $O(n)$ recursion depth/accumulator, plus space for all generated strings

Approach (short): Maintain the string built so far (usually as a list for $O(1)$ append/pop) plus any counters needed to check validity (e.g., open-paren count). At each step, try each legal next character given the current counters, recurse, then pop it off before trying the next option.

Keywords & Constraints: "generate all valid...", "balanced parentheses", "letter combinations", "binary strings with constraint", $n \leq 15\text{-}20$ for output size to remain manageable.

```
def generate_parentheses(n):
    """All combinations of n pairs of balanced parentheses."""
    result, current = [], []
    def backtrack(open_count, close_count):
        if len(current) == 2 * n:
            result.append("".join(current))
            return
        if open_count < n:
            current.append('(')
            backtrack(open_count + 1, close_count)
            current.pop()
        if close_count < open_count:
            current.append(')')
            backtrack(open_count, close_count + 1)
            current.pop()
    backtrack(0, 0)
    return result

def letter_combinations(digits):
    """All letter combinations a phone-digit string could represent."""
    if not digits:
        return []
    mapping = {'2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
               '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'}
    result, current = [], []
    def backtrack(i):
        if i == len(digits):
            result.append("".join(current))
            return
        for ch in mapping[digits[i]]:
            current.append(ch)
```



```

        backtrack(i + 1)
        current.pop()
    backtrack(0)
    return result

def add_operators(nums, target):
    """Insert +, -, * between digits of `nums` so the expression evaluates to target."""
    result = []
    n = len(nums)
    def backtrack(i, expr, value, last_operand):
        if i == n:
            if value == target:
                result.append(expr)
            return
        for j in range(i, n):
            piece = nums[i:j + 1]
            if len(piece) > 1 and piece[0] == '0':
                break
            num = int(piece)
            if i == 0:
                backtrack(j + 1, piece, num, num)
            else:
                backtrack(j + 1, expr + '+' + piece, value + num, num)
                backtrack(j + 1, expr + '-' + piece, value - num, -num)
                backtrack(j + 1, expr + '*' + piece, value - last_operand + last_operand * num, last_operand * num)
    backtrack(0, "", 0, 0)
    return result

```

Practice Questions — String Backtracking

Q1. Generate Parentheses Generate all combinations of n pairs of valid balanced parentheses. *Approach:* Track open/close counts; only add '(' if open < n and only add ')' if close < open, pruning invalid prefixes as they're built rather than checking validity at the end.

Q2. Letter Combinations of a Phone Number Return all letter combinations a digit string could represent. *Approach:* Branch over the mapped letters for the current digit, recurse to the next digit position, and pop before trying the next letter.

Q3. Restore IP Addresses Insert dots to form all valid IP addresses from a digit string. *Approach:* Build the address piece by piece, trying 1-3 digit segments at each step and pruning segments that are out of range or have invalid leading zeros.

Q4. Expression Add Operators Insert +, -, * between digits so the expression evaluates to a target. *Approach:* Backtrack over every possible next number boundary and every operator choice, carrying enough state (running value and last operand) to correctly undo multiplication's different precedence.

Q5. Palindrome Permutation (Generate All) Generate all distinct palindromic permutations of a string's characters. *Approach:* Build only the first half via permutation-style backtracking over character counts, then mirror it — pruning characters with odd leftover counts appropriately.

Q6. Generate All Binary Strings with No Two Consecutive 1s Generate all length- n binary strings with no "11" substring. *Approach:* Branch over appending '0' (always allowed) or '1' (only allowed if the previous character wasn't '1'), demonstrating a constraint that depends on just the immediately preceding choice.

2.8 Pruning, Bounding & Bitmask Optimization

What it's for: A collection of techniques layered on top of any backtracking template to make it fast enough to run in practice: cutting branches early with cheap checks, comparing against a running best (bounding), and representing "used/available" state as bits for $O(1)$ set operations instead of $O(n)$ loops or sets.

Use Cases:

- Speeding up N-Queens/Sudoku-style problems using bitmasks for attacked columns/diagonals.
- Branch-and-bound optimization problems (e.g., prune a branch once its best-possible outcome can't beat the current best answer).
- Traveling Salesman / Hamiltonian path problems where visited-set state fits in an integer bitmask ($n \leq \sim 20$).
- Any backtracking solution that times out without these optimizations despite being logically correct.

Limitations:

- Bitmask tricks only work cleanly when n is small enough to fit in machine-word-sized integers (practically $n \leq 20$ -25 for a single int bitmask).
- Bounding requires a cheap, valid upper/lower bound estimate — a bad bound either prunes too little (no speedup) or, if incorrect, can prune away valid answers.
- These are optimizations, not a different algorithmic paradigm — always get a correct (if slow) backtracking solution first, then optimize.

Time Complexity: Same worst-case complexity class as the underlying backtracking, but with a much smaller practical constant/search space **Space Complexity:** $O(1)$ extra for bitmask state (vs $O(n)$ for set-based tracking); $O(n)$ for bounding's "best so far" tracking

Approach (short): Identify what "used/available" state is checked on every call and represent it as bits (used $|= 1 \ll i$ to mark, used $\& (1 \ll i)$ to check, used $\wedge= 1 \ll i$ to unmark). Separately, track the best complete answer found so far and skip exploring a branch once you can prove it cannot improve on that best.

Keywords & Constraints: " $n \leq 20$ with subset/visited state", "TSP", "Hamiltonian path", "branch and bound", "minimize/maximize with backtracking", any backtracking problem that times out with a naive set/array for visited tracking.

```
def n_queens_count_bitmask(n):
    """N-Queens solution count using bitmasks for columns and both diagonals -- much faster than sets."""
    count = 0
    full_mask = (1 << n) - 1

    def backtrack(row, cols, diag1, diag2):
        nonlocal count
        if row == n:
            count += 1
            return
        available = full_mask & ~(cols | diag1 | diag2)      # bits set where a queen CAN go
        while available:
            pos = available & (-available)                  # extract lowest set bit (one candidate column)
            available -= pos                                # remove it from remaining candidates
            backtrack(row + 1, cols | pos, (diag1 | pos) << 1, (diag2 | pos) >> 1)

    backtrack(0, 0, 0, 0)
    return count

def tsp_held_karp_style_backtrack(dist, n):
    """Traveling Salesman via backtracking + bitmask visited set + pruning against best found so far."""
    best = [float('inf')]

    def backtrack(pos, visited_mask, cost, count):
        if cost >= best[0]:
            return                                           # BOUND: can't possibly beat current best, prune
        if count == n:
            best[0] = cost
```

```

        best[0] = min(best[0], cost + dist[pos][0])        # complete the tour back to start
        return
    for nxt in range(n):
        if not (visited_mask & (1 << nxt)):                # nxt not yet visited
            backtrack(nxt, visited_mask | (1 << nxt), cost + dist[pos][nxt], count + 1)

    backtrack(0, 1, 0, 1)                                # start at city 0, mark it visited
    return best[0]

def subsets_bitmask(nums):
    """All subsets enumerated directly via bitmask iteration (no explicit choose/unchoose recursion needed)."""
    n = len(nums)
    result = []
    for mask in range(1 << n):                            # every integer 0..2^n-1 IS a subset encoding
        subset = [nums[i] for i in range(n) if mask & (1 << i)]
        result.append(subset)
    return result

```

Practice Questions — Pruning, Bounding & Bitmask Optimization

Q1. N-Queens with Bitmask Count/solve N-Queens using bitmask state instead of sets. *Approach:* Represent attacked columns and both diagonals as integers; use bitwise AND/OR/shift to compute available positions in $O(1)$ per row instead of $O(n)$ set lookups.

Q2. Traveling Salesman (Backtracking + Bound) Find the minimum-cost tour visiting all cities once. *Approach:* Track visited cities in a bitmask, and prune any partial tour whose cost already meets or exceeds the best complete tour found so far (branch and bound).

Q3. Hamiltonian Path Existence Determine if a path exists visiting every vertex exactly once. *Approach:* Backtrack over next-vertex choices with a bitmask visited set; the small n typical of this problem (≤ 20) makes bitmask state tracking both correct and fast.

Q4. Sudoku Solver with Bitmask Candidates Speed up Sudoku solving using bitmasks of possible digits per row/col/box. *Approach:* Precompute a bitmask of used digits per row, column, and box; combine with bitwise OR and negate to get the mask of legal candidates for a cell in $O(1)$ instead of scanning 1-9 with row/col/box loops.

Q5. Maximum Score by Assigning Tasks (Branch and Bound) Assign tasks to workers to maximize total score, tasks/workers small. *Approach:* Backtrack over assignments, bounding each branch by an optimistic best-possible remaining score and pruning once that bound can't beat the current best full assignment.

Q6. Subsets via Bitmask Iteration Generate all subsets of a set without explicit choose/unchoose recursion. *Approach:* Loop mask from 0 to $2^n - 1$; each mask's set bits directly encode one subset — shows backtracking's output can sometimes be produced without recursion at all once state fits in a bitmask.

PART 3: QUICK REFERENCE CHEAT SHEET

Pattern	Purpose	Time	Space	Watch Out For
Subsets & Combinations	Include/exclude or choose-k	$O(2^n \times n) / O(C(n,k) \times k)$	$O(n)$	Duplicates need sort + same-depth skip
Permutations	All orderings	$O(n! \times n)$	$O(n)$	Must track used[] and un-mark on backtrack
Combination Sum	Reach a target sum	Exponential, pruned	$O(\text{target}/\text{min})$	Same idx = reuse allowed; idx+1 = no reuse
Partitioning	Split into valid pieces	$O(2^n \times n)$	$O(n)$	Precompute validity (e.g. palindrome table) to save time
Grid/Maze Backtracking	Explore paths in a grid	$O(rc \times 4^L)$	$O(L)$	Must unmark visited cells on backtrack
Constraint Satisfaction	Fill board under constraints	Pruning-dependent	$O(n)$	Track attacked rows/cols/diagonals for $O(1)$ checks
String Backtracking	Build strings under rules	$O(2^n)$ to $O(\text{Catalan}(n))$	$O(n)$	Prune invalid prefixes DURING construction, not after
Pruning/Bounding/Bitmask	Speed up any of the above	Same class, smaller constant	$O(1)$ for bitmask	Bitmask only viable for $n \leq \sim 20-25$

Decision Guide — “Which Pattern Do I Use?”

1. Need every possible selection of elements, order doesn't matter? → Subsets & Combinations
2. Need every possible ORDERING of elements? → Permutations
3. Need selections that hit an exact target sum, with reuse allowed or not? → Combination Sum
4. Need to split a sequence into contiguous pieces that each satisfy a property? → Partitioning
5. Problem lives on a 2D grid and involves moving between cells? → Grid / Maze Backtracking
6. Filling a structured board where each placement must satisfy row/column/region rules? → Constraint Satisfaction (N-Queens, Sudoku)
7. Building a string/expression character-by-character under formation rules? → String Backtracking
8. A correct backtracking solution is too slow, and n is small ($\leq \sim 20-25$) or you need to compare against a running best? → Add Pruning, Bounding, or a Bitmask
9. Only need ONE optimal number/count (not every explicit solution) and subproblems overlap? → That is Dynamic Programming — see the dedicated DP handbook